



Assignment 2

Semester Project Title: AI Resume Analyzer

Student Details

	Student Name	Student Reg #	Student Degree
Student-1	Aazeb Ali Malik	2023003	BSCSW
Student-2	Abdullah Ihsan	2023039	BSCSW
Student-3	Faseeh	2023689	BSCSW

1: Class Identification

The system is organized into a three-layer architecture: **Presentation/UI**, **Application/Domain**, and **Persistence/Data** layer, ensuring separation of concerns.

1. Presentation Layer (User Interface & Controllers)

AuthController:

- **Purpose:** Manages user registration and login requests.
- **Responsibilities:** Handle login and register requests, validate input, and issue authentication tokens.
- **Key Attributes:** AuthService dependency.
- **Key Methods:** login(credentials), register(user_data), logout()
- **Relationships:** Collaborates with AuthService.

ResumeController:

- **Purpose:** Manages resume-related interactions.
- **Responsibilities:** Handle upload, view, and delete resume requests. Trigger analysis when a job description is matched.
- **Key Attributes:** ResumeService dependency.
- **Key Methods:** uploadResume(file), getResume(id), deleteResume(id), triggerAnalysis(job_id, resume_id)
- **Relationships:** Collaborates with ResumeService.

JobController:

- **Purpose:** Manages job description interactions (Recruiter).
- **Responsibilities:** Handle post, edit, and search job descriptions. Retrieve candidate shortlists.
- **Key Attributes:** JobService dependency.
- **Key Methods:** postJob(data), editJob(id, data), searchCandidates(job_id)
- **Relationships:** Collaborates with JobService.

2. Application/Domain Layer (Business Logic & Core Services)

User

- **Purpose:** Represents a user (Candidate, Recruiter, Admin).
- **Responsibilities:** Hold basic user information.
- **Key Attributes:** id, username, password_hash, email, role (Enum), created_at
- **Key Methods:** verifyPassword(input)
- **Relationships:** Associated with Resume, JobDescription.

Resume

- **Purpose:** Represents a candidate's uploaded resume document.
- **Responsibilities:** Store document details and extracted text content.
- **Key Attributes:** id, user_id, filename, file_path, extracted_text, upload_date
- **Key Methods:** extractText(file), getAnalysisHistory()
- **Relationships:** Associated with User, AnalysisReport.

JobDescription

- **Purpose:** Represents a job posting provided by a recruiter.

- **Responsibilities:** Store job posting details.
- **Key Attributes:** id, recruiter_id, title, description_text, keywords (List), posted_date
- **Key Methods:** extractKeywords()
- **Relationships:** Associated with User, AnalysisReport.

AnalysisReport

- **Purpose:** Stores the result of a single resume-job analysis.
- **Responsibilities:** Store match score, feedback, and keyword metrics.
- **Key Attributes:** id, resume_id, job_id, match_score (Float), missing_skills (List), matched_keywords (List), analysis_date
- **Key Methods:** getScore(), generateFeedback()
- **Relationships:** Associated with Resume, JobDescription.

AnalysisService

- **Purpose:** Orchestrates the AI/NLP processing.
- **Responsibilities:** Handle the core business logic of resume analysis, delegating to the NLPModel.
- **Key Attributes:** NLPModel dependency, ReportRepository dependency.
- **Key Methods:** runAnalysis(resume, job_desc), saveReport(report)
- **Relationships:** Collaborates with NLPModel, ReportRepository.

NLPModel

- **Purpose: (Abstract/Interface)** Defines the contract for any AI model used for similarity checking.
- **Responsibilities:** Provides the core NLP functionality (semantic similarity).
- **Key Attributes:** None
- **Key Methods:** analyze(text1, text2), extractFeatures(text)
- **Relationships:** Implemented by TensorFlowNLPModel.

3. Persistence Layer (Data Access)

UserRepository

- **Purpose:** Handles all database operations for User objects.
- **Responsibilities:** Retrieve, save, update, and delete user records in the PostgreSQL database.
- **Key Attributes:** db_connection
- **Key Methods:** findById(id), findByUsername(username), save(user), delete(id)
- **Relationships:** Collaborates with the PostgreSQL database.

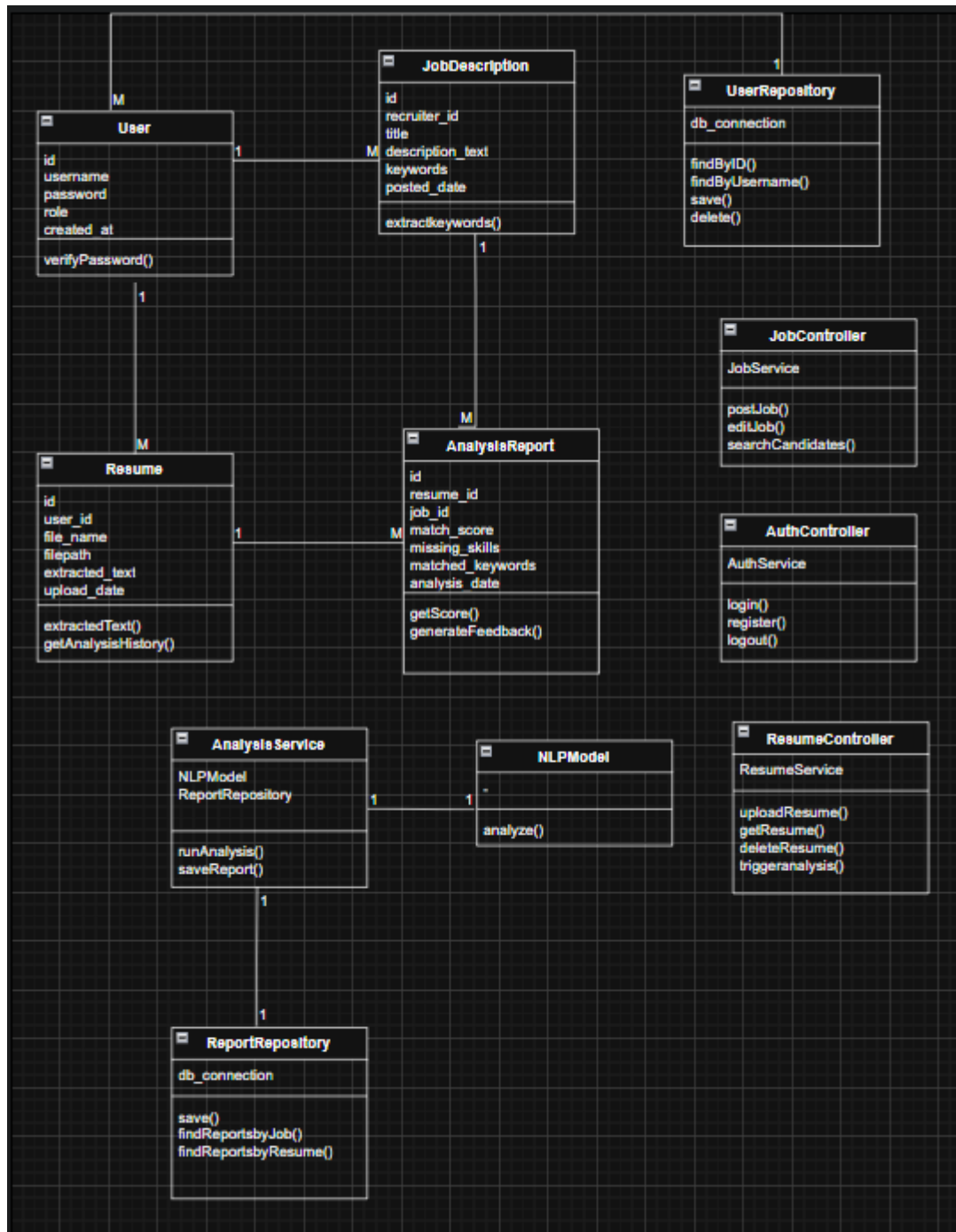
ReportRepository

- **Purpose:** Handles all database operations for AnalysisReport objects.
- **Responsibilities:** Retrieve, save, and query reports.
- **Key Attributes:** db_connection
- **Key Methods:** save(report), findReportsByJob(job_id), findReportsByResume(resume_id)
- **Relationships:** Collaborates with the PostgreSQL database.

2: CRC Cards (Conceptual Design)

Class Name	Responsibilities	Collaborators
User	1. Manage personal identification details (ID, role, credentials). 2. Request authentication and authorization.	AuthController, AuthService
Resume	1. Store the textual and file content of a CV. 2. Track its analysis history. 3. Provide text content to AnalysisService.	ResumeController, AnalysisReport
JobDescription	1. Store the job description text and associated keywords. 2. Provide the job text and requirements to AnalysisService.	JobController, AnalysisReport, NLPModel
AnalysisReport	1. Hold the match score and actionable feedback. 2. Allow retrieval by both Resume and Job ID.	AnalysisService, ReportRepository
AnalysisService	1. Orchestrate the comparison process. 2. Receive Resume and Job data. 3. Delegate scoring to NLPModel. 4. Package and persist the final AnalysisReport.	NLPModel, AnalysisReport, ReportRepository
NLPModel	1. Calculate semantic similarity between two texts. 2. Identify missing keywords based on features.	AnalysisService
UserRepository	1. Persist (Save/Update/Delete) User objects to the database. 2. Retrieve User objects by ID or username.	AuthService
ReportRepository	1. Persist (Save/Update) AnalysisReport objects to the database. 2. Query reports based on job_id or resume_id.	AnalysisService

3: UML Class Diagram (Technical Design)



4: Design Patterns Planning

Pattern Name	Pattern Type	Applied To / Module	Problem Solved	Why This Pattern?
Factory Method	Creational	NLPModel instantiation within AnalysisService.	Avoids direct dependency on concrete AI model classes (e.g., TensorFlowNLPModel or PyTorchNLPModel).	Allows the system to easily swap or introduce new AI/NLP backends without changing the core AnalysisService logic, supporting the Open/Closed Principle (OCP) .
Singleton	Creational	Database connection or Configuration Manager.	Ensures that only one instance of the costly database connection pool or global configuration state exists.	Optimizes resource usage and prevents synchronization issues with critical shared resources like the db_connection.
Adapter	Structural	Integrating third-party resume parsing library.	Allows a pre-existing third-party resume parsing library (with an incompatible interface) to be used by the ResumeService.	Decouples the system from external library interfaces, making future library replacements seamless.
Decorator	Structural	AnalysisService to add caching or logging.	Adds cross-cutting concerns (like result caching or detailed logging) to the analysis process dynamically without modifying the original AnalysisService class.	Provides flexible, runtime-added functionality while adhering to the Open/Closed Principle (OCP) .
Strategy	Behavioral	Keyword extraction and scoring algorithms within NLPModel.	Allows different scoring algorithms (e.g., TF-IDF vs. Semantic Embedding) to be interchangeable at runtime.	Enables dynamic selection of the best-performing algorithm based on system load or recruiter preference, supporting future improvements without core modification.
Observer	Behavioral	ResumeService notification upon new AnalysisReport.	Automatically notifies interested parties (e.g., the RecruiterDashboard or CandidateUI) when a new AnalysisReport is generated.	Decouples the analysis result creation from its display, ensuring real-time updates across different parts of the application.

5: Design Principles and SOLID Compliance

(a) OOAD Design Principles

Abstraction The system uses abstraction heavily in the Persistence and Analysis layers. The NLPModel interface (or abstract class) hides the complex implementation details of the underlying AI model (TensorFlow, PyTorch, etc.) from the AnalysisService. The service only knows it needs to call an analyze(text1, text2) method, not *how* that method is implemented.

Encapsulation Classes like User, Resume, and JobDescription encapsulate their data (attributes) by making them private and exposing them only through controlled access methods (getters/setters, or behavior-driven methods like Resume.extractText()). This protects data integrity. For instance, the raw password_hash in the User class is protected and only accessed/modified via AuthService methods.

Inheritance Inheritance is used in the Persistence Layer. An abstract AbstractRepository class could define common CRUD operations (save(), findById()) which are then inherited and specialized by concrete classes like UserRepository and ReportRepository. Similarly, the TensorFlowNLPModel would inherit or implement the abstract NLPModel.

Polymorphism This is achieved through the NLPModel interface. The AnalysisService can hold a reference to the abstract type NLPModel but at runtime, it can operate on any concrete implementation (e.g., TensorFlowNLPModel or DummyNLPModel) by calling the same method: model.analyze(resume, job_desc). This allows the system to be run with different scoring engines without code changes.

Cohesion & Coupling **High Cohesion** is ensured by the layered architecture. For example, UserRepository is only responsible for managing User data (functional cohesion). Low Coupling is enforced by using interfaces and dependencies; the ResumeController only depends on the ResumeService interface, not its concrete implementation, minimizing inter-module reliance.

(b) SOLID Principles

Principle	Where Applied	How It's Ensured
SRP (Single Responsibility Principle)	AnalysisService vs. NLPModel	AnalysisService handles the business workflow (orchestration, reporting, saving). NLPModel handles only the AI/NLP algorithms (scoring, feature extraction).
OCP (Open/Closed Principle)	NLPModel Interface	The system is Open for extension by creating a new PyTorchNLPModel implementation (extending the NLPModel interface) to add a new analysis method. It is Closed for modification, as the AnalysisService doesn't need to change to use the new model (via the Factory Method).
LSP (Liskov Substitution Principle)	UserRepository and ReportRepository	Both repository classes can be substituted for an abstract Repository reference in a testing environment (e.g., using a MockUserRepository) without breaking the system's database access logic.
ISP (Interface Segregation Principle)	Single Repository vs. Specific Repositories	Instead of a single, massive Repository, specific repository interfaces (e.g., UserRepository, ReportRepository) are used. This prevents clients (e.g.,

		AuthService) from depending on methods they don't use (like report-specific queries).
DIP (Dependency Inversion Principle)	Controllers and Services	High-level modules (Controllers) depend on abstractions (Service Interfaces), not concrete implementations (Services). This is typically achieved through dependency injection, where the ResumeController is injected with an abstract ResumeService.